Project 1.1 Building a Sequential MapReduce Library in Go

Overview

In this project, you will build a simple MapReduce library to gain hands-on experience with the Go programming language and understand the fundamentals of MapReduce.
This project will serve as the foundation for Project 1.2, where you will extend this implementation to a distributed version.

Project 1.1 is structured into two parts:

- Part A: Implement the core sequential MapReduce functions.
- Part B: Develop a simple MapReduce application by implementing a word counter.

The Go Programming Language

All assignments in this course are implemented in Go, so we recommend getting familier with the language before moving on.
Follow the instructions from https://go.dev/doc/install to install Go on your machine's platform.

The following are some reference materials to get you started:

- Effective Go
- Go by Example
- *The Go Programming Language* (recommeded book)

One of Go's standout features is its built-in support for asynchronous programming and concurrency. If you come from a C++ / Python background and haven't used threads/asyncio before, this might seem like a foreign concept. We will talk about basic concepts of asynchronous programming and Go's concurreny model during the first TA session, but for those who cannot make it here are the materials:

- Asynchronous Go for Beginners
- Concurrency is not Parallelism

Project Setup

- Run `tar -zxvf lab1_1_mapreduce.tgz` to extract and setup the project environment

- There should be the following in `lab1_1_mapreduce/`
  - `go.mod`
  - `main/`
  - `mapreduce/`
- Run `go run main/wc.go` from the project root
  - If you see `Word Counter: see usage comments in file`, then everything is setup correctly and you're ready to start the project.

Part A: Implementing Sequential MapReduce

Objective

Complete the core functions to enable sequential MapReduce processing. This mode processes tasks one at a time and is useful for debugging before moving to a distributed version.

Tasks

Modify and complete the following files:

- `mapreduce/common_map.go`: Implement `doMap()`
- `mapreduce/common_reduce.go`: Implement `doReduce()`

Workflow of Sequential MapReduce:

1. The master assigns input files to **map** tasks
2. Each **map** task:
   - reads its input file
   - applies the `map` function
   - write intermediate key/value pairs into multiple files (one per reduce task)
3. The master collects intermediate results and assigns **reduce** tasks
4. Each **reduce** task:
   - collects intermediate files
   - applies the `reduce` function to aggregate results
   - write final output

Testing

Run the provided Go test suite by `go test -race -run Sequential ./mapreduce`. Test cases that have passed will have an `ok` next to the test names.

For verbose output enable dugging by setting `debugEnabled = true` in `mapreduce/common.go`, and add the `-v` flag (for verbose) in the above command (i.e., `go test -race -run Sequential -v ./mapreduce`).

Part B: Implementing a Word Counter Using MapReduce

Objective

Now that you have a working sequential MapReduce framework, use it to implement a **word count** application.

Tasks

Modify the following file:

- `main/wc.go`: Implement `mapF()` and `reduceF()`

Function Requirements:

- `mapF(file string, contents string)[]mapreduce.KeyValue`
  - Splits the input text into words (using `unicode.IsLetter`).
  - Outputs key/value pairs where:
    - Key = word
    - Value = occurrence count (always `"1"` in this phase).
- `reduceF(key string, values []string) string`
  - Sums the occurrences of each word
  - Outputs the final count as a string

Testing

Test your word counter by:

- `go run main/wc.go master sequential main/pg-huckleberry_finn.txt`
- All test files: `bash main/test-wc.sh`

Submission Instructions

1. Move to the parent directory of the project folder
   - e.g., if the project folder is located at `~/lab1_1_mapreduce`, then `cd` to `~`
2. Run `tar -zcvf lab1_1_mapreduce_<GROUP_NUMBER>.tgz lab1_1_mapreduce`

- make sure that `mapreduce/common_map.go`, `mapreduce/common_reduce.go`, `main/wc.go` are included

3. Do **not** leave debugging print statements in your code
4. Only **one** member of the group has to submit to Canvas

Grading

1. The TA's will run `tar -zxvf lab1_1_mapreduce_<GROUP_NUMBER>.tgz` to extract submissions
   - check your submission by running the above command
   - the output should be the ***project folder*** (i.e. `lab1_1_mapreduce`) and not individual files

2. We will grade parts A & B of this project separately:
   - Part A:
     - Each test case will be run twice—first without checking for race conditions, and then with race condition checks enabled
     - e.g., for case `TestSequentialSingle`, we will run both
       - `go test -run SequentialSingle ./mapreduce`
       - `go test -race -run SequentialSingle ./mapreduce`
     - A test case that passes both runs will receive 100% of the score. If it fails the race condition check, 30% will be deducted. Failing both runs results in a score of 0%.

Test Case	TA cmd	`ok`	`race`	`fail`
`TestSequentialSingle`	`SequentialSingle`	2pt	1.4pt	0pt
`TestSequentialMany`	`SequentialMany`	2pt	1.4pt	0pt

   - Part B:

Test Case	TA cmd	`ok`	`fail`
`word count`	`bash main/test-wc.sh`	4pt	0pt

3. All members in the group (indicated by `<GROUP_NUMBER>`) will receive the same grade.